

EE345 Mod4-Lec1: Classification via Logistic Regression

[VMLS, 14]

Classification

MNIST

- **So far....**: goal was to predict a real-valued outcome y from some data x
- **Module 4** (Classification): the outcome y takes on only a finite number of values, and hence is sometimes called a **label**, or in statistics, a **categorical**.

Binary Classification

- As before, the goal is to predict the outcome y from the feature vector x .
- The classifier f is a function that takes in a feature vector x and outputs a prediction $f(x)$.
- But now the outcome y takes on only two values:

typically $\{-1, +1\}$ or $\{0, 1\}$

$y \in \mathbb{R}$

From Regression to Classification

- So far: **least squares** for regression problems
 $y \approx \theta^T x, \quad y \in \mathbb{R}.$
 $\phi(x) = x$
 $\begin{bmatrix} \text{user} \\ \text{ad} \end{bmatrix} = x$
- Now: **binary classification**
 $y \in \{0, 1\}$ SPAM/HAM, click/no-click

Problem: linear model $x^T \theta$. WANT TO MAP

$$\text{score}(x) \in [0, 1] \quad \text{or} \quad p(y=1 | x) \in [0, 1]$$

- Idea: keep a linear score, then pass it through a *squashing* function to get a probability.

Classification Examples

- **Email spam detection.**

- ▶ *Feature vector:* $x \in \mathbb{R}^n$ contains features of an email message like word counts etc.
- ▶ *Outcome:* $y = +1$ if an email represented by feature vector x is **SPAM** and -1 otherwise.

- **Fraud detection.**

- ▶ *Feature vector:* $x \in \mathbb{R}^n$ contains features associated with a credit card user such as average monthly spending, median prices of purchases over last week, etc.
- ▶ *Outcome:* $y = +1$ for **fraudulent transactions**, and -1 otherwise.

- **Document Classification.**

- ▶ *Feature vector:* $x \in \mathbb{R}^n$ is a word count (or histogram) vector for a document
- ▶ *Outcome:* $y = +1$ if the document has some topic (e.g., politics) and -1 otherwise

Two Linear Approaches to Binary Classification

- We consider $y \in \{-1, +1\}$ and a linear score:

$$s(x) = \theta^\top x.$$

- Two ways to turn scores into predictions:
 - ▶ **Least Squares Classification:** choose θ to make $s(x_i)$ close to y_i . ✓
 - ▶ **Logistic Regression:** choose θ to model $p(y_i = 1 \mid x_i)$ via logistic function. ✓

Method 1: Least Squares Classification

Least Squares Classifier

- Least squares classifier: this is a simple method that works well in many cases
- do ordinary real-valued least squares fitting of the outcome, ignoring that $y \in \{-1, +1\}$
- i.e., choose basis functions $\phi_1, \dots, \phi_p$ and solve $\theta^T x$

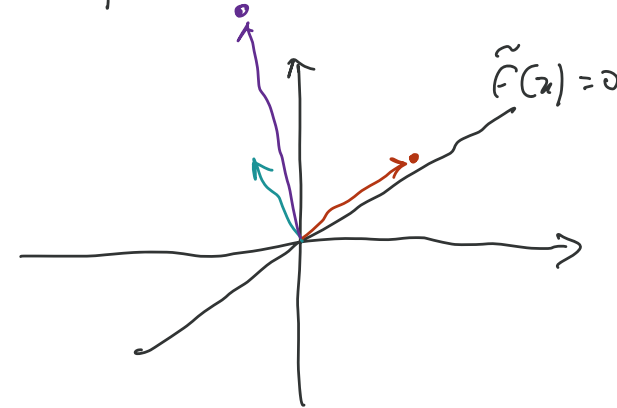
$$\min_{\theta \in \mathbb{R}^p} \left\{ \sum_{i=1}^N (y_i - \tilde{f}(x_i))^2 \mid \tilde{f}(x_i) = \theta_1 \phi_1(x_i) + \dots + \theta_p \phi_p(x_i), i = 1, \dots, N \right\}$$

$\tilde{f}$ is the least square fit model

- final classifier (least squares classifier):

$$\hat{f}(x) = \underbrace{\text{sign}(\tilde{f}(x))}_{\text{sign}(\theta^T x)} = \begin{cases} +1 & \text{if } \tilde{f}(x) \geq 0 \\ -1 & \text{if } \tilde{f}(x) < 0 \end{cases}$$

$\tilde{f}(x) = \theta^T x$



Intuition for Least Squares Classifier

- The value $\tilde{f}(x)$ is a number "near" $+1$ when $y = +1$ and near -1 when $y = -1$
- Forced to guess one of the two possible outcomes, $\text{sign}(\tilde{f}(x))$ is a good choice—it is the nearest neighbor of $\tilde{f}(x)$ among $\{-1, +1\}$
- $\tilde{f}(x)$ also tells us our [↓]confidence[↑] in our assignment

Least Squares Classification

- If we pick $\phi_1(x) = 1$ and $\phi_2(x) = x$, then the problem is

$$\min_{\theta} \|X\theta - y\|_2^2, \quad \text{where } y_i \in \{-1, +1\}.$$

- Closed-form solution:

$$\left[\hat{\theta}_{\text{LS}} = (X^T X)^{-1} X^T y. \right] \rightarrow \hat{f}(x) = \hat{\theta}_{\text{LS}}^T x$$

- Prediction rule:

$$\hat{y}(x) = \text{sign}(\hat{\theta}_{\text{LS}}^T x) = \begin{cases} +1 & \hat{\theta}_{\text{LS}}^T x \geq 0 \\ -1 & \hat{\theta}_{\text{LS}}^T x < 0 \end{cases}$$

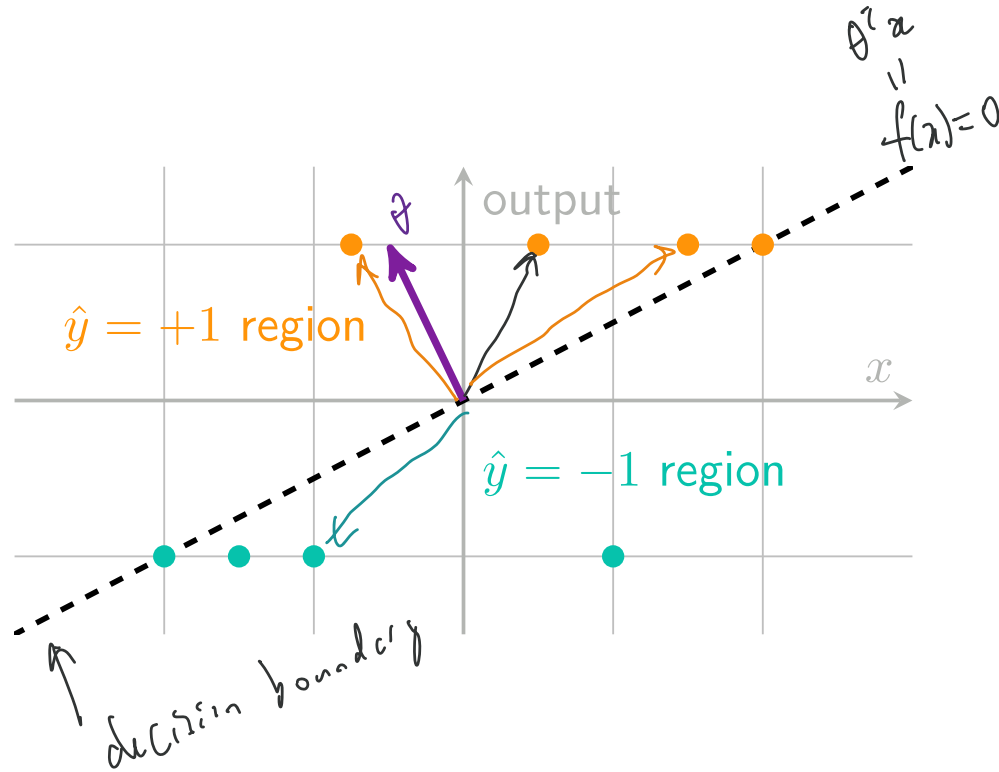
- Pros:

- ▶ Simple, fast, closed-form.
- ▶ Works surprisingly well when classes are linearly separable.

- Cons:

- ▶ Squared error is not made for classification.
- ▶ Large errors are overly penalized (outliers).
- ▶ Predicted scores are not probabilities.

Least Squares Classifier with the Sign Function



- Map labels to $\{-1, +1\}$ and fit:

$$\hat{\theta} = \arg \min_{\theta} \sum_i (y_i - \theta_0 - \theta_1 x_i)^2.$$

- Prediction is a **real-valued** output

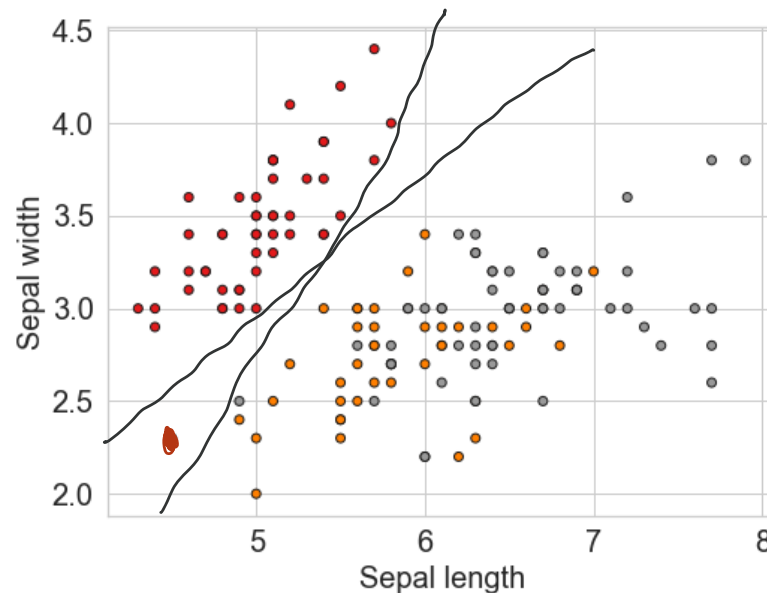
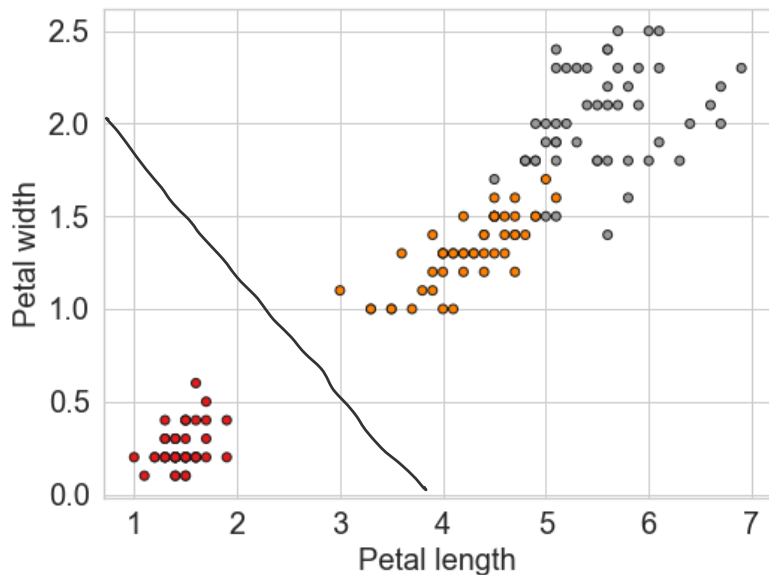
$$f(x) = \theta_0 + x \theta_1$$

- Turn it into a classifier with

$$g(x) = \text{sign}(f(x))$$

Example: Predicting Iris Type

Three types of iris — setosa, versicolour, virginica — 50 samples of each type



four features:

- x_1 sepal length [cm], x_2 sepal width [cm]
- x_3 petal length [cm], x_4 petal width [cm]
- Goal: build classifier to detect if iris is virginica or not

Iris Data Set Example: Confusion Matrix

data Matrix:

$$\theta_0 + \bar{\theta}^T x$$

$$X = \begin{bmatrix} 1 & x_1^{(1)} & x_2^{(1)} & x_3^{(1)} & x_4^{(1)} \\ 1 & x_1^{(2)} & x_2^{(2)} & x_3^{(2)} & x_4^{(2)} \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & x_1^{(N)} & x_2^{(N)} & x_3^{(N)} & x_4^{(N)} \end{bmatrix}$$

Step 2: generate your classifier

$$\text{Sign}(\bar{\theta}_{LS}^T x) = \begin{cases} +1, & \text{if } \bar{\theta}_{LS}^T x \geq 0 \\ -1, & \text{if } \bar{\theta}_{LS}^T x < 0 \end{cases}$$

labels : $y = (\underbrace{-1, \dots, -1}_{100}, \underbrace{+1, \dots, +1}_{50})$

Step 1:

$$\min_{\theta} \|X\theta - y\|^2$$

Find $\hat{\theta}_{LS} = \underset{\theta}{\operatorname{argmin}} \|X\theta - y\|^2$



$$\hat{\theta} = (X^T X)^{-1} X^T y, \quad \hat{y}_i = \text{sign}(\hat{\theta}^T x^{(i)})$$

Both Methods:

Step 1: minimize loss fn on data to generate $\hat{\theta}$ model parameters

Step 2: Apply transformation fn. to map to label

Method 2: Logistic Regression

Least Sq. Class.

Step 1: $\min \|X\theta - y\|^2$

Step 2: $\text{sign}(\)$

Log. Regression

Step 1: $\min \text{Log-likelihood}$

Step 2: Sigmoid fn.

Logistic Regression: Fixing the Problems

$$Y_i \in \{0, 1\} \quad X_i$$

$$Y \in \{-1, +1\}$$

$$p(Y=1 | X) = \sigma(\theta^T X) = \frac{1}{1 + \exp(-\theta^T X)}$$

- output is in $[0, 1]$
- smooth transition between classes

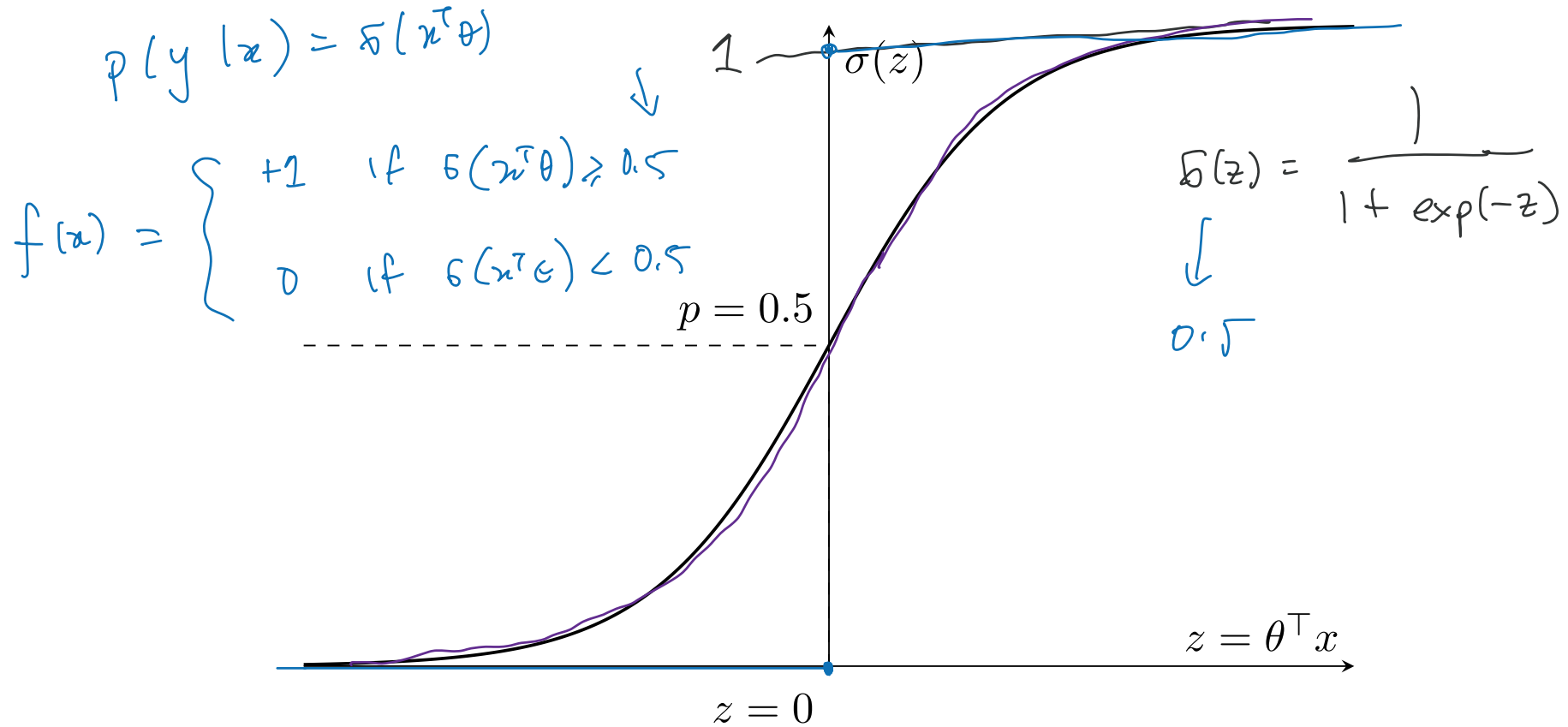
Decision Boundary

Because the sigmoid is monotone:

$$p(y = 1 \mid x) = 0.5 \iff \theta^\top x = 0.$$

Same boundary as linear regression classification, BUT better calibrated probabilities.

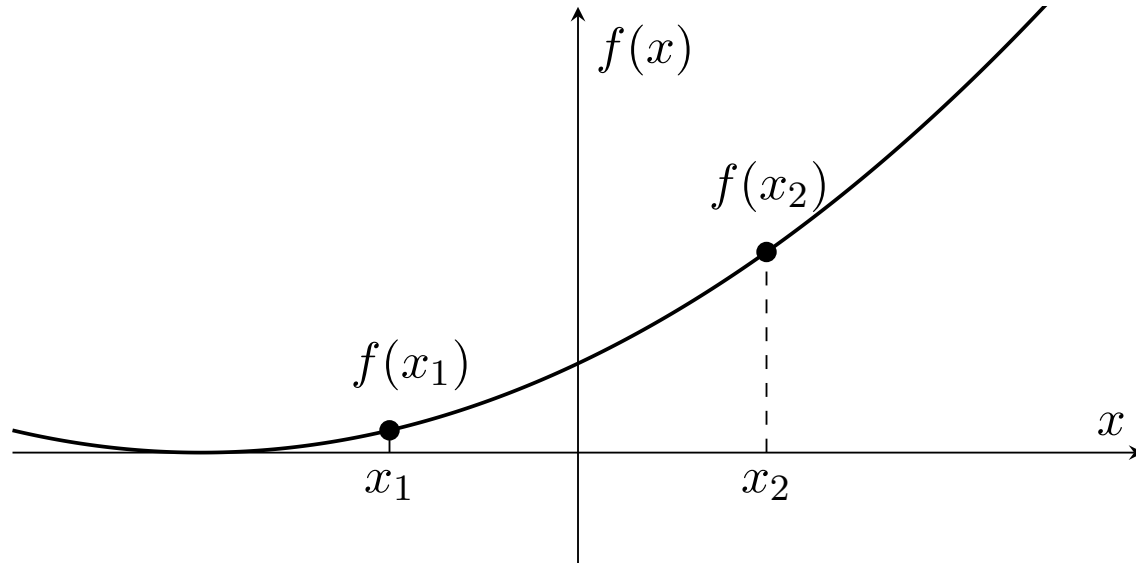
Sigmoid



The linear score $z = \theta^T x$ is mapped through the sigmoid to a probability $\sigma(z) \in (0, 1)$;
 $z = 0$ corresponds to $p = 0.5$.

Sigmoid Properties: Monotone functions are order-preserving

A function $f : \mathbb{R} \rightarrow \mathbb{R}$ is monotone increasing if $x_1 < x_2 \Rightarrow f(x_1) \leq f(x_2)$



Checking Monotonicity Using the Derivative

Monotone increasing: if $f \in C^1$ (i.e. continuous and differentiable)

$$f'(x) \geq 0 \quad \forall x \quad \Leftrightarrow \quad x_1 < x_2 \quad \Rightarrow \quad f(x_1) \leq f(x_2)$$

Strictly monotone increasing: $f'(x) > 0$ for all x

Monotone decreasing: $f'(x) \leq 0$ for all x

Derivative Test: Necessary and Sufficient (for C^1)

Assume: $f : \mathbb{R} \rightarrow \mathbb{R}$ is continuously differentiable (i.e., $f \in C^1$).

Sufficient:

$$f'(x) \geq 0 \quad \forall x \quad \implies \quad f \text{ is monotone non-decreasing.}$$

Necessary: If f is monotone non-decreasing, then for every x ,

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \geq 0.$$

Conclusion: For C^1 (and therefore C^2) functions,

$$f \text{ monotone increasing} \quad \iff \quad f'(x) \geq 0 \quad \forall x.$$

Note: Without differentiability, monotonicity does *not* require f' to exist.

Monotonicity of the Sigmoid

$$\underline{x^T \theta = z}$$

The logistic (sigmoid) function is

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad \underbrace{\sigma'(z) = \sigma(z)(1 - \sigma(z))}$$

$$\mathbb{1}\{\sum p \geq 0.5\} = \begin{cases} 1 & \text{if } p \geq 0.5 \\ 0 & \text{o.w.} \end{cases}$$

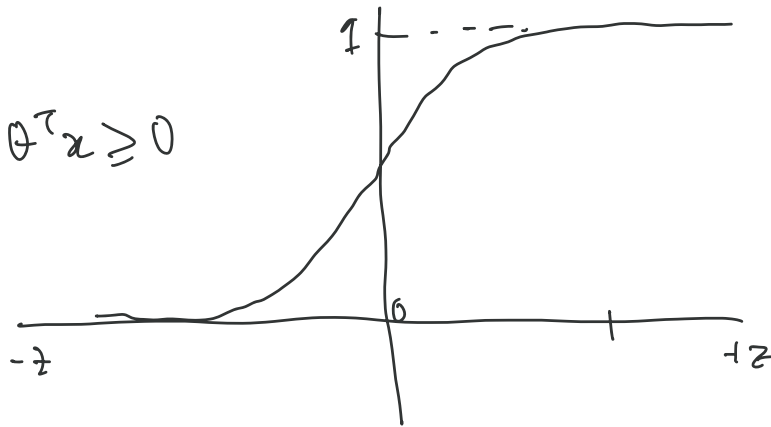
$$\bullet \quad \underbrace{0 < \sigma(z) < 1 \quad \forall z \Rightarrow 0 < \sigma'(z) < 1 \quad \forall z}$$

$\Rightarrow$ then for $\sigma(z)$ is a monotone increasing fn.
 $z_1 < z_2 \Rightarrow \sigma(z_1) < \sigma(z_2)$

• Threshold:

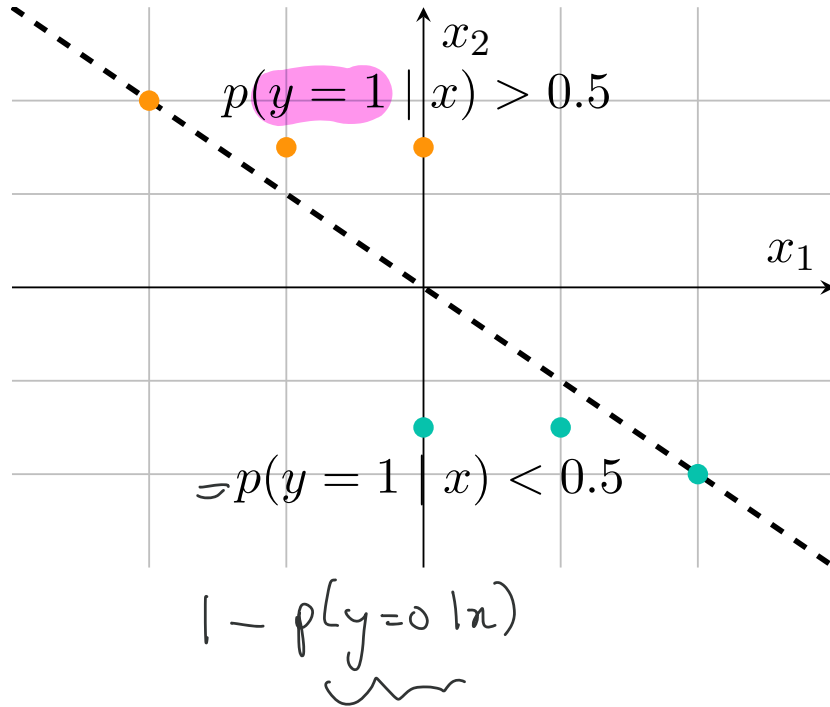
$$p(y=1 | x) = \sigma(x^T \theta) \geq 0.5 \Leftrightarrow \theta^T x \geq 0$$

• Thus the decision bdd. for $\hat{y} = \mathbb{1}\{p \geq 0.5\}$
 is the hyperplane $\theta^T x = 0$



Decision Boundary for Logistic Regression

$$\underline{p(y = 1 | x) = \sigma(\theta^T x)}.$$

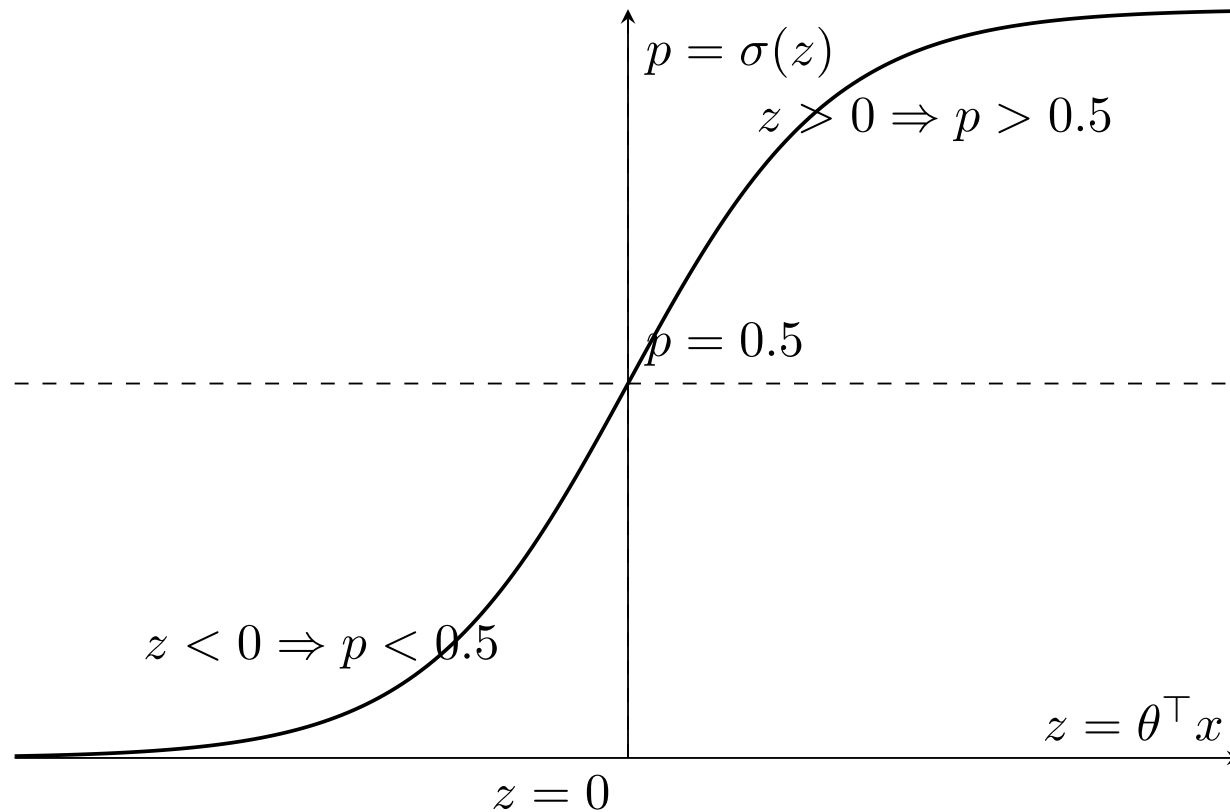


Decision Rule:

$$\hat{y}(x) = \mathbb{1}_{\{p(y=1|x) \geq 0.5\}}$$

$$\hat{y}(x) = \begin{cases} 1 & \text{if } \sigma(\theta^T x) \geq 0.5 \\ 0 & \text{if } \sigma(\theta^T x) < 0.5 \end{cases}$$

Sigmoid: Mapping $\theta^\top x$ to Probability



Because σ is strictly increasing and $\sigma(0) = 0.5$,

$$\theta^\top x > 0 \implies \sigma(\theta^\top x) > 0.5, \quad \theta^\top x < 0 \implies \sigma(\theta^\top x) < 0.5.$$

Why Can Sigmoid Outputs Be Interpreted as Probabilities?

Odds and Log-Odds

Odds

For a probability $p \in (0, 1)$, the *odds* of the event are

Log-odds (logit)

In logistic regression,

Interpreting Coefficients via Odds Ratios

Consider a single feature x and model

Why Can Sigmoid Outputs Be Interpreted as Probabilities?

- Logistic regression assumes a probabilistic model:

$$y_i \sim \text{Bernoulli}(p_i), \quad p_i = P(y_i = 1 \mid x_i, \theta).$$

- We model the **log-odds** as linear:

$$\log \frac{p_i}{1 - p_i} = \theta^\top x_i.$$

- Solving for p_i gives the sigmoid:

$$p_i = \sigma(\theta^\top x_i) = \frac{1}{1 + e^{-\theta^\top x_i}}.$$

- Therefore $\sigma(\theta^\top x)$ is the **model's estimate** of the probability that $y = 1$.
- **Calibration:** These probabilities may or may not be well-calibrated depending on the dataset and training.

Fitting the Logistic Regression Model

Summary so far: Logistic Regression

- Model a probability:

$$p(y = 1 \mid x) = \sigma(\theta^\top x) = \frac{1}{1 + e^{-\theta^\top x}}.$$

- Prediction rule:

$$\hat{y}(x) = \begin{cases} 1, & p(x) \geq 0.5, \\ 0, & p(x) < 0.5. \end{cases}$$

- Outputs calibrated probabilities.
- **How do we fit the model?**

Intuition: Cost function via Likelihood

Likelihood: A score of how well our model explains the data we observed.

- High likelihood = the model "expected" the data.
- Low likelihood = the model is "surprised."

For classification: If the true label is $y \in \{0, 1\}$ and the model predicts p , we want:

$$y = 1 \implies p \text{ large}, \quad y = 0 \implies (1 - p) \text{ large}.$$

Why log likelihood? Turning products into sums makes optimization easier and more stable. Negative log likelihood becomes a loss function.

Cross entropy loss:

$$\ell = -(y \log p + (1 - y) \log(1 - p))$$

Cross-entropy punishes confident wrong predictions very strongly, and rewards confident correct predictions.

Likelihood and Loss for Logistic Regression

- Data: (x_i, y_i) for $i = 1, \dots, n$, with $y_i \in \{0, 1\}$.

- Model:

$$p_i := p(y_i = 1 \mid x_i) = \sigma(\theta^\top x_i).$$

- Likelihood (under independence):
- Log-likelihood:
- We **maximize** $\ell(\theta)$, or equivalently **minimize** the negative log-likelihood:

Loss Function: Negative Log-Likelihood or Cross-Entropy Loss

Maximum likelihood $\implies$ minimize logistic loss:

$$\ell(\theta) = - \sum_{i=1}^n \left[y_i \log(\sigma(\theta^\top x_i)) + (1 - y_i) \log(1 - \sigma(\theta^\top x_i)) \right].$$

Properties:

- Convex in θ .
- No closed form $\longrightarrow$ use gradient descent.

Gradient of the Logistic Loss

- Recall $p_i = \sigma(\theta^\top x_i)$ and

$$J(\theta) = - \sum_{i=1}^n \left(y_i \log p_i + (1 - y_i) \log(1 - p_i) \right).$$

- Using $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, one can show

$$\nabla_{\theta} J(\theta) = \sum_{i=1}^n (p_i - y_i) x_i.$$

- In matrix notation: let $X \in \mathbb{R}^{n \times d}$ have rows $x_i^\top$, $p \in \mathbb{R}^n$ have entries p_i , and $y \in \mathbb{R}^n$ have entries y_i . Then

$$\nabla_{\theta} J(\theta) = X^\top (p - y).$$

Gradient of the Logistic Loss: Detailed Derivation (1)

Gradient of the Logistic Loss: Detailed Derivation (2)

Gradient of the Logistic Loss: Detailed Derivation (3)

Gradient of the Logistic Loss: Detailed Derivation (4)

Gradient Descent for Logistic Regression

Logistic Regression with Labels $\{-1, +1\}$

With labels $y_i \in \{-1, +1\}$, the logistic model can be written in a clean, symmetric form.

Model:

$$p(y_i \mid x_i; \theta) = \sigma(y_i \theta^\top x_i), \quad \text{where } \sigma(z) = \frac{1}{1 + e^{-z}}.$$

Interpretation:

- If $y_i = +1$, we want $\theta^\top x_i$ to be large and positive
- If $y_i = -1$, we want $\theta^\top x_i$ to be large and negative
- Both cases are captured by $y_i \theta^\top x_i$

Negative log-likelihood:

$$\ell(\theta) = \sum_{i=1}^n \log \left(1 + \exp \left(-y_i \theta^\top x_i \right) \right).$$

This form is simpler than the $\{0, 1\}$ expression, which requires two separate terms:
 $y \log p + (1 - y) \log(1 - p)$.

Simpler Gradients with Labels $\{-1, +1\}$

For logistic regression with loss

$$\ell(\theta) = \sum_{i=1}^n \log \left(1 + \exp \left(-y_i \theta^\top x_i \right) \right),$$

the gradient has a simple unified form.

Derivative with respect to θ :

$$\nabla_{\theta} \ell(\theta) = \sum_{i=1}^n \left(\frac{1}{1 + \exp(y_i \theta^\top x_i)} \right) (-y_i x_i).$$

Simpler Gradients with Labels $\{-1, +1\}$

Using the sigmoid identity

$$\sigma(-y_i \theta^\top x_i) = \frac{1}{1 + \exp(y_i \theta^\top x_i)},$$

the gradient becomes

$$\nabla_{\theta} \ell(\theta) = \sum_{i=1}^n -y_i x_i \sigma(-y_i \theta^\top x_i)$$

Key advantages:

- One unified formula (no separate $y = 1$ / $y = 0$ cases)
- Symmetric treatment of the two classes
- Clean geometry: pushing $y_i \theta^\top x_i$ to be large and positive

Regularized Logistic Regression

- To control overfitting, we can add an ℓ_2 penalty:

$$J_\lambda(\theta) = - \sum_{i=1}^n \left(y_i \log p_i + (1 - y_i) \log(1 - p_i) \right) + \lambda \|\theta\|_2^2.$$

- Gradient becomes

$$\nabla_\theta J_\lambda(\theta) = X^\top (p - y) + 2\lambda\theta.$$

- Gradient descent update:

$$\theta^{(t+1)} = \theta^{(t)} - \eta \left(X^\top (p^{(t)} - y) + 2\lambda\theta^{(t)} \right).$$

- Same idea as ridge regression, but with logistic loss.

Next Time: Prediction Errors: Model Performance Metrics